

INWK6312 - Lab 1

Linux Foundations and Version Control

Contents

Introduction	2
Lab Objectives	2
Lab Environment and Preparation	2
Part A: Linux and Filesystem Foundations	3
Task 0: Connecting to Your Lab VM	3
Task 1: Creating Filesystem Structure	3
Task 2: Text Processing Toolkit	4
Task 3: systemd Service Management	5
Task 4: IP Address and Link Management with iproute2	6
Task 5: Network Namespaces and Virtual Wires	6
Task 6: Building a Mini Routed Topology with Namespaces	7
Task 7: Automating Infrastructure with Bash	9
Part B: Version Control and Environment Management	12
Task 8: Initializing the Root Repository and Environment	12
Task 9: The First Commit	12
Task 10: Branching for Safe Changes	13
Task 11: Simulating and Resolving a Merge Conflict	14
Task 12: Publishing to GitHub and Opening a Pull Request	15
Clean Up	16
Submission	16
Appendix: Command Summary	17
Linux and Networking	17
Git and Version Control	17

Introduction

This lab introduces the fundamental Linux and version control skills required for the remainder of the course. You will work directly on an Ubuntu virtual machine to build a routed network using Linux namespaces, which serves as a manual precursor to the containerized emulation used in later modules. Instead of treating this as a standalone exercise, you will establish a professional repository structure that will house all your work, shared tools, and network topologies for Labs 2 through 5.

Lab Objectives

By the end of this lab, you will be able to:

1. Navigate the Linux filesystem and establish a multi-module project hierarchy.
2. Manage file ownership and permissions for automation scripts.
3. Use text processing tools like **grep** and **sed** to manipulate configuration data.
4. Manage **systemd** services through their full lifecycle.
5. Configure isolated network environments using namespaces, veth pairs, and static routing.
6. Initialize a centralized Python virtual environment and a global Git repository.
7. Implement a professional Git workflow including branching, merge conflict resolution, and pull requests.

Lab Environment and Preparation

You will need:

- Your assigned Ubuntu VM IP address provided in Brightspace: _____.
- A GitHub account.
- The GitHub Classroom assignment link, provided by an email from the instructor.

If any of the components above are missing, check with your lab instructor before starting the lab.

Throughout this document, commands you type are shown in code blocks. Where a command needs elevated privileges it is shown with **sudo**. If a step does not show **sudo**, you should not need it.

Part A: Linux and Filesystem Foundations

Task 0: Connecting to Your Lab VM

1. Use an SSH client to log in to your assigned VM using the provided credentials. Possible SSH clients include: Command Prompt, PuTTY, or MobaXTerm on Windows, or the built in Terminal app on macOS and Linux. Start the terminal, and use SSH to login in to the assigned VM:

```
ssh student@<VM IP Address>
```

2. Enter the password Meilab123.
3. Practice basic text editing with nano by creating a file and saving changes.

```
nano welcome.txt
```

Save and exit using **Ctrl+O**, **Enter**, **Ctrl+X**. You can also **Ctrl-X** directly then **Y**, **Enter** at the prompt.

4. Practice with vim to understand its modal nature, focusing on switching between Normal and Insert modes.

```
vim welcome.txt
```

Press **i** to enter Insert mode, make a change by adding few lines of text, press **Esc** to return to Normal mode.

5. Familiarize yourself with navigation keys (**h**, **j**, **k**, **l**) and basic commands like **dd** for line deletion and **u** for undo. Save and quit with **:wq**, **Enter**.

Task 1: Creating Filesystem Structure

Objective: Establish a root level directory that will serve as the single source of truth for the entire course.

1. Create a root directory named **labs** in your home folder.

```
mkdir ~/labs
```

2. Within this root, you will create a subfolder structure that separates lab assignments from shared utilities and network definitions.

```
mkdir -p ~/labs/lab1/scripts ~/labs/tools ~/labs/topology
```

The **lab1** directory will hold your specific deliverables for this module, the **tools** directory will house shared utilities like device libraries used across all labs, and the **topology** directory will store your persistent network definitions starting in Lab 2.

3. Practice moving between these directories using **cd** and confirming your location with **pwd**.

```
cd ~/labs  
pwd
```

-
4. Create a test file in `lab1` then identify the owner, the group, and the three permission triplets, owner, group, and other.

```
touch lab1/sample.txt
ls -l lab1/sample.txt
```

5. Use `chmod` to restrict permissions so only your user has read and write access.

```
chmod 600 lab1/sample.txt
ls -l lab1/sample.txt
```

Questions and Deliverables

1. What is the purpose of the `~` in the steps 1 and 2 above?
2. What does the option `-p` do in step 2?
3. Show file permissions after step 5.

Task 2: Text Processing Toolkit

Objective: Use Linux utilities to extract and modify data within configuration files.

1. Navigate to `~/labs/lab1` and create a file named `sample-config.txt` containing several interface and IP address lines (do not cut-paste the last EOF).

```
cat > sample-config.txt << 'EOF'
hostname R1
interface GigabitEthernet0/0
description Uplink to Core
ip address 10.1.1.1 255.255.255.0
interface GigabitEthernet0/1
description Access VLAN10
ip address 10.1.10.1 255.255.255.0
interface GigabitEthernet0/2
description Access VLAN20
ip address 10.1.20.1 255.255.255.0
router ospf 1
network 10.1.1.0 0.0.0.255 area 0
network 10.1.10.0 0.0.0.255 area 0
EOF
```

2. View the whole file.

```
cat sample-config.txt
```

3. Use `grep` to filter for specific interface configuration blocks.

```
grep "interface" sample-config.txt
```

4. Use a pipe to send `grep` output into `cut` to extract only the IP addresses from the file.

```
grep "ip address" sample-config.txt | cut -d ' ' -f 3
```

5. Use **sed** to perform a search and replace operation, such as changing a VLAN ID, and apply the change permanently using the **-i** flag.

```
cat sample-config.txt
sed -i 's/VLAN10/VLAN30/' sample-config.txt
cat sample-config.txt
```

Questions and Deliverables

1. Provide the modified file `sample-config.txt`
2. Describe, briefly, the general application of each tools used in this task.

Task 3: systemd Service Management

Objective: control a service through its full lifecycle without affecting the SSH session you are currently using for connectivity.

1. Create a systemd unit file `inwk-demo.service` under `/etc/systemd/system/`. Note that because this directory is owned by the root user, you must use `sudo` to create and edit the file.

```
sudo nano /etc/systemd/system/inwk-demo.service
```

Enter the following content, save, and exit.

```
[Unit]
Description=INWK6312 Demo Service

[Service]
ExecStart=/usr/bin/sleep infinity
Restart=always

[Install]
WantedBy=multi-user.target
```

2. Save the file. Reload the **systemd** daemon to pick up the new configuration:

```
sudo systemctl daemon-reload
```

3. Start the service and check its current status: `bash` `sudo systemctl start inwk-demo`
`sudo systemctl status inwk-demo`

4. Enable the service so it automatically starts when the VM boots:

```
sudo systemctl enable inwk-demo
```

5. Practice stopping the service and observing the difference between an inactive status and a failed state.

6. Use `journalctl` to inspect the logs generated by your service.

```
journalctl -u inwk-demo
```

Questions and Deliverables

1. Provide the output of steps 3, 5, and 6 above.

Task 4: IP Address and Link Management with iproute2

Objective: use the `ip` command to inspect and modify the VM network configuration, which serves as the base for all future automation.

1. List all network interfaces on the VM and identify which ones are in the UP state.

```
ip link show
```

2. View the IPv4 addresses assigned to each interface.

```
ip addr show
```

3. Inspect the routing table to identify the default gateway.

```
ip route show
```

4. View the neighbor table to see the learned MAC addresses on the local segment:

```
ip neighbor show
```

5. Add a secondary IP address to the loopback interface, verify it is present, and then remove it to practice basic interface manipulation.

```
sudo ip addr add 192.0.2.1/24 dev lo
ip addr show lo
sudo ip addr del 192.0.2.1/24 dev lo
ip addr show lo
```

6. View active listening sockets and identify which services are currently accepting connections on the VM.

```
ss -tulpn
```

7. Test reachability to an external host using `mtr -c 3`, which provides a combined view of path latency and hop information.

```
ping -c 3 8.8.8.8
mtr -c 3 8.8.8.8
```

Questions and Deliverables

1. What is the purpose of the option `-c 3` in step 7? What will happen if it is removed?
2. Provide the output of the `mtr` command from step 7.

Task 5: Network Namespaces and Virtual Wires

Objective: build two fully isolated network stacks on the same VM and connect them with a virtual patch cable, known as a veth pair.

1. Create two namespaces named **ns-red** and **ns-blue** using `sudo ip netns add`.

```
sudo ip netns add ns-red
sudo ip netns add ns-blue
ip netns list
```

2. Create a veth pair to act as your virtual wire. Name the ends **veth-red** and **veth-blue**.

```
sudo ip link add veth-red type veth peer name veth-blue
```

3. Move each end of the wire into its respective namespace.

```
sudo ip link set veth-red netns ns-red
sudo ip link set veth-blue netns ns-blue
```

4. Verify the move by checking the interfaces on the host. The veth ends should no longer appear in the default namespace.

```
ip link show
sudo ip netns exec ns-red ip link show
sudo ip netns exec ns-blue ip link show
```

5. Assign the IP address 10.10.1.1/24 to **veth-red** and 10.10.1.2/24 to **veth-blue**.

```
sudo ip netns exec ns-red ip addr add 10.10.10.1/24 dev veth-red
sudo ip netns exec ns-blue ip addr add 10.10.10.2/24 dev veth-blue
```

6. Bring both veth interfaces and the loopback interface UP within each namespace using `ip netns exec`.

```
sudo ip netns exec ns-red ip link set veth-red up
sudo ip netns exec ns-red ip link set lo up
sudo ip netns exec ns-blue ip link set veth-blue up
sudo ip netns exec ns-blue ip link set lo up
```

7. Verify connectivity by pinging between the namespaces.

```
sudo ip netns exec ns-red ping -c 3 10.10.10.2
```

Questions and Deliverables

1. Provide the output of the verification commands from step 3. Which namespace's output no longer lists **veth-red** and **veth-blue** on the host itself, and why not.
2. Provide the output of the ping test in step 7.

Task 6: Building a Mini Routed Topology with Namespaces

Objective: extend Task 5 into a three namespace topology consisting of two hosts and a router to observe how a Linux router forwards traffic between subnets.

Target topology: **ns-hostA** (10.10.1.2/24) -- **ns-router** -- **ns-hostB** (10.10.2.2/24).

1. Clean up the namespaces from the previous task, then create three fresh ones.

```
sudo ip netns del ns-red
sudo ip netns del ns-blue
sudo ip netns add ns-hostA
sudo ip netns add ns-router
sudo ip netns add ns-hostB
```

2. Create the first veth pair between ns-hostA and ns-router. Name the ends veth-a and veth-r1.

```
sudo ip link add veth-a type veth peer name veth-r1
sudo ip link set veth-a netns ns-hostA
sudo ip link set veth-r1 netns ns-router
```

3. Create the second veth pair between ns-router and ns-hostB. Name the ends veth-r2 and veth-b.

```
sudo ip link add veth-r2 type veth peer name veth-b
sudo ip link set veth-r2 netns ns-router
sudo ip link set veth-b netns ns-hostB
```

4. Move the interfaces into their respective namespaces and verify the placement. ns-hostA should contain veth-a, ns-router should contain both veth-r1 and veth-r2, and ns-hostB should contain veth-b.

```
for ns in ns-hostA ns-router ns-hostB; do
    echo "-- $ns --"
    sudo ip netns exec $ns ip link show
done
```

5. Assign the following IP addresses.

```
sudo ip netns exec ns-hostA ip addr add 10.10.1.2/24 dev veth-a
sudo ip netns exec ns-router ip addr add 10.10.1.1/24 dev veth-r1
sudo ip netns exec ns-router ip addr add 10.10.2.1/24 dev veth-r2
sudo ip netns exec ns-hostB ip addr add 10.10.2.2/24 dev veth-b
```

6. Bring every interface and loopback up in all three namespaces.

```
for ns in ns-hostA ns-router ns-hostB; do
    sudo ip netns exec $ns ip link set lo up
done
sudo ip netns exec ns-hostA ip link set veth-a up
sudo ip netns exec ns-router ip link set veth-r1 up
sudo ip netns exec ns-router ip link set veth-r2 up
sudo ip netns exec ns-hostB ip link set veth-b up
```

Verify every interface is up and carries the address you expect, before moving on to routing.

```
for ns in ns-hostA ns-router ns-hostB; do
    echo "-- $ns --"
```



```
sudo ip netns exec $ns ip addr show
done
```

7. Enable IP forwarding inside **ns-router**. This is the critical kernel setting that allows the namespace to pass traffic between its interfaces.

```
sudo ip netns exec ns-router sysctl -w net.ipv4.ip_forward=1
```

8. Add static routes on the hosts.

```
sudo ip netns exec ns-hostA ip route add 10.10.2.0/24 via 10.10.1.1
sudo ip netns exec ns-hostB ip route add 10.10.1.0/24 via 10.10.2.1
```

9. Test end to end connectivity by pinging from **ns-hostA** to **ns-hostB**.

```
sudo ip netns exec ns-hostA ping -c 3 10.10.2.2
```

Questions and Deliverables

1. Provide the output of the forwarding verification command from step 8, and the output of the ping test in step 9.
2. In your own words, explain what would have happened in step 9 if IP forwarding had not been enabled on **ns-router**, even though the static routes on **ns-hostA** and **ns-hostB** were both correctly configured.

Task 7: Automating Infrastructure with Bash

Objective: turn the manual steps from Task 6 into repeatable scripts to avoid manual operational pitfalls.

1. Navigate to `~/labs/lab1/scripts`.
2. Create a script named `build-topology.sh`. Use the logic from Task 6 to automate the creation of namespaces, veth pairs, IP assignments, and routing.

```
nano build-topology.sh
```

3. Start from this skeleton, then extend it yourself to complete the full topology from Task 6, including the veth pairs, IP addresses, interfaces up, IP forwarding, and static routes.

```
#!/bin/bash
# build-topology.sh
# Builds the INWK6312 Lab 1 routed namespace topology:
# ns-hostA -- ns-router -- ns-hostB

set -e

NAMESPACES=("ns-hostA" "ns-router" "ns-hostB")

for ns in "${NAMESPACES[@]}; do
    if ip netns list | grep -q "$ns"; then
```

```

        echo "$ns already exists, skipping"
    else
        ip netns add "$ns"
        echo "Created $ns"
    fi
done

# TODO: create the veth pairs and move each end into the right namespace
# TODO: assign addresses to each interface
# TODO: bring up every interface and every loopback
# TODO: enable IP forwarding on ns-router
# TODO: add the static routes on ns-hostA and ns-hostB

echo "Topology build complete"

```

4. Create a companion script named `teardown-topology.sh` that deletes the three namespaces using a loop.

```

nano teardown-topology.sh

#!/bin/bash
# teardown-topology.sh

NAMESPACES=("ns-hostA" "ns-router" "ns-hostB")

for ns in "${NAMESPACES[@]}; do
    if ip netns list | grep -q "$ns"; then
        ip netns del "$ns"
        echo "Deleted $ns"
    else
        echo "$ns does not exist, skipping"
    fi
done

```

5. Make both scripts executable using `chmod +x`.

```

chmod +x build-topology.sh
chmod +x teardown-topology.sh

```

6. Run `teardown-topology.sh` to clean your environment, then run `build-topology.sh` and verify connectivity with a ping.

```

sudo ./teardown-topology.sh
sudo ./build-topology.sh

```

Questions and Deliverables

1. Provide the completed scripts generated on previous steps.
2. Provide the output confirming the teardown script removed all three namespaces in step 5,

and the output confirming the topology was rebuilt and reachable again in step 6.

Part B: Version Control and Environment Management

Task 8: Initializing the Root Repository and Environment

Objective: establish the root of your project as a Git repository and set up a unified Python environment that will serve the entire course.

1. Move into your root `~/labs` directory. This directory will be the root of your Git repository and your Python environment. `bash` `cd ~/labs`
2. Initialize the repository and set the main branch name. `bash` `git init` `git branch -m main`
3. Configure your Git identity. Use your own name and email address.

```
git config --global user.name "Your Name"
git config --global user.email "your_email@dal.ca"
```

4. Create a global `.gitignore` file in `~/labs`. This file must proactively exclude the following to keep your repository clean:

```
cat > .gitignore << 'EOF'
# your Python virtual environment
.velab/
# Python bytecode
__pycache__/_
# sensitive token files used in later labs
.envr.
# directories auto-generated by Containerlab
**/clab-*/
EOF
```

5. Initialize a single Python virtual environment at the root:

```
python3 -m venv .velab
```

6. Activate the environment. You will use this same environment for every lab in this course.

```
source ~/labs/.velab/bin/activate
```

7. Create a root level `requirements.txt` file. This file will be updated cumulatively as new libraries are introduced in future modules.

```
touch ~/labs/requirements.txt
```

Task 9: The First Commit

Objective: bring your Part A work under version control and practice the basic Git lifecycle.

1. Run `git status` and observe that your `lab1`, `.gitignore`, and `requirements.txt` are listed as untracked, while `.velab` is not showing.

```
git status
```

2. Stage your work for the first commit.

```
git add lab1/ .gitignore requirements.txt
```

3. Verify the change in status. The files should now be listed as changes to be committed.

```
git status
```

4. Perform your initial commit.

```
git commit -m "Initial commit: Lab 1 foundations and root repository  
↪ structure"
```

5. Review your commit history using `git log`. `bash git log` `git log --oneline`

Task 10: Branching for Safe Changes

Objective: isolate infrastructure changes on a feature branch before merging them into the main line of development.

1. Create and switch to a feature branch named `feature/add-hostC` in one step and verify.

```
git switch -c feature/add-hostC  
git branch
```

2. Edit the `build-topology.sh` and `teardown-topology.sh` files to add a third host named `ns-hostC`. Connect `ns-hostC` to `ns-router` on a new subnet `10.10.3.0/24`. Update the `nameservers` array, add the new `veth` pair, and ensure the routing logic and forwarding are correctly extended.

3. Run your scripts to verify that `ns-hostA` can ping the new `ns-hostC`.

```
sudo ~/labs/lab1/scripts/teardown-topology.sh  
sudo ~/labs/lab1/scripts/build-topology.sh  
sudo ip netns exec ns-hostC ping -c 3 10.10.1.2
```

4. Stage and commit your changes on the feature branch.

```
git add lab1/scripts/build-topology.sh lab1/scripts/teardown-topology.sh  
git commit -m "Add ns-hostC to the topology"
```

5. Switch back to the main branch `bash git switch main`

6. Confirm that the changes for `ns-hostC` are not present in your scripts while on `main`, demonstrating branch isolation.

```
cat lab1/scripts/build-topology.sh
```

7. Merge the feature branch into `main`. Since `main` has not changed, this will be a fast forward merge.

```
git merge feature/add-hostC
```

Questions and Deliverables

1. Provide the output of `git branch` from step 1, confirming which branch was active.
2. In step 5, `cat scripts/build-topology.sh` on main should not show `ns-hostC` yet. Provide that output, and explain in one sentence why the change is not visible there.

Task 11: Simulating and Resolving a Merge Conflict

Objective: resolve a conflict manually when two different branches modify the same configuration lines in the same file.

1. Ensure you are on the main branch, then create a branch named `experiment-a`:

```
bash git switch main  
git switch -c experiment-a
```
2. In `~/labs/lab1/scripts/build-topology.sh`, change the subnet used for `ns-hostA` from `10.10.1.0/24` to `10.10.11.0/24`.

```
nano ~/labs/lab1/scripts/build-topology.sh
```

3. Commit this change.

```
git add lab1/scripts/build-topology.sh  
git commit -m "Renumber ns-hostA subnet to 10.10.11.0/24"
```

4. Return to main and create a second branch named `experiment-b`.

```
git switch main  
git switch -c experiment-b
```

5. Repeat step 2, but use `10.10.21.0/24` instead. Commit this change.

```
git add lab1/scripts/build-topology.sh  
git commit -m "Renumber ns-hostA subnet to 10.10.21.0/24"
```

6. Return to main and merge `experiment-a`. This merge should be clean.

```
git switch main  
git merge experiment-a
```

7. Attempt to merge `experiment-b` into main. Git will report a merge conflict because both branches modified the same lines.

```
git merge experiment-b
```

8. Open `lab1/scripts/build-topology.sh` and look for the conflict markers: `<<<<<<<`, `=====`, and `>>>>>>`. Manually edit the file to keep the `10.10.21.0/24` subnet and remove all conflict markers.
9. Stage the resolved file and complete the merge commit.

```
git add lab1/scripts/build-topology.sh
git commit
```

10. Confirm the topology still works after resolving the conflict.

```
sudo ./lab1/scripts/teardown-topology.sh
sudo ./lab1/scripts/build-topology.sh
```

11. Visualize the branch and merge history.

```
git log --graph --oneline --all
```

Task 12: Publishing to GitHub and Opening a Pull Request

Objective: connect your local repository to GitHub Classroom and complete a peer review cycle using a pull request.

1. Open the GitHub Classroom assignment link provided by your instructor and accept it to generate your personal remote repository.
2. Generate a personal access token on GitHub with `repo` scope. Use this token as your password when prompted during the push process.
3. Link your local `~/labs` repository to the new remote.

```
cd ~/labs
git remote add origin <your-classroom-repo-url>
```

4. Push your local `main` branch to GitHub.

```
git push -u origin main
```

5. Create a new branch named `docs/readme` for project documentation.

```
git switch -c docs/readme
```

6. Create a `~/labs/README.md` file at the root. Describe the contents of the `lab1`, directories, and explain how to run the topology scripts.

```
nano README.md
```

7. Stage, commit, and push the `docs/readme` branch to GitHub.

```
git add README.md
git commit -m "Add README describing lab1 folder content"
git push -u origin docs/readme
```

8. On the GitHub website, open a pull request from `docs/readme` into `main`.
9. Review your own pull request by adding at least one inline comment on the diff to highlight a specific implementation detail.
10. Merge the pull request using the GitHub web interface.

-
11. Return to your local terminal, switch to `main`, and pull the merged changes: `git pull`.

```
git switch main
git pull
```

Clean Up

You should leave the `inwk-demo` systemd service running as it is a harmless background process. You may leave your namespace topology active or use your `teardown-topology.sh` script to remove it, as the scripts allow you to rebuild the infrastructure at any time.

Do not delete the `.velab` directory or your `.gitignore` file, as these are central to the unified environment you will use for the rest of the course. Do not delete your GitHub Classroom repository, as you will continue to extend this same repository in Lab 2 and beyond.

Submission

Before submitting, verify that your repository history includes the following deliverables:

1. The initial commit containing `build-topology.sh`, `teardown-topology.sh`, and `sample-config.txt`.
2. The root level `.gitignore` and `requirements.txt` files.
3. The `feature/add-hostC` merge.
4. The resolved merge conflict from Task 11, visible as a merge commit in the git graph.
5. The `REFLECTION.md` file in the `lab1` folder.
6. The `docs/readme` pull request, merged and visible on your GitHub Classroom repository.

Submit the link to your GitHub repository as instructed by your course platform.

Appendix: Command Summary

Linux and Networking

Command	Usage
pwd, ls -l, cd	Navigate and inspect the filesystem
chmod, chown	Change permissions and ownership
cat, grep, cut, sed	View and process text
systemctl start/stop/status/enable	Control a systemd service
journalctl -u <service>	Read logs for a specific service
ip link show / set	View and change interface state
ip addr show / add / del	View and change IP addresses
ip route show / add	View and change the routing table
ip neighbor show	View the ARP/neighbor cache
ip netns add / del / list / exec	Manage network namespaces
ip link add type veth peer name	Create a veth pair

Git and Version Control

Command	Usage
git init	Create a new local repository
git config --global	Set commit identity
git status	Show staged and unstaged changes
git diff / --staged	View changes
git add	Stage changes
git commit -m	Record a snapshot
git log --graph --oneline	View history
git switch -c	Create and switch branches
git restore	Discard or unstage changes
git merge	Combine branch histories
git remote add origin	Link to a remote repository
git push -u origin	Publish changes to the remote